# 12. Testing, Table Tests, Fuzz, and Benchmarks

## 12. Testing, Table Tests, Fuzz, and Benchmarks

**Phase:** Phase 2 — Core language depth
**Estimated time:** 1–2 days
**Slug:** `12-testing`

### Learning objectives

- Write table-driven tests
- Use subtests with t.Run
- Introduce fuzz targets and benchmarks
- Treat tests as executable specifications

### Prerequisites

Complete prior numbered lessons. Type every example yourself — reading Go is not writing Go.

### Why this matters

Tests are executable specs. Table-driven tests, fuzzing, and benchmarks turn Go's fast compiler into a design tool.

This lesson is a building block for later modules in the Goodway path. If you only skim, the exercises will feel like busywork; if you type the examples and narrate them aloud, the same patterns will reappear in HTTP handlers, concurrency, and AI service glue.

### Core concepts

- Table tests + t.Run give clear failure names.
- Fuzzing feeds random inputs to find edge cases.
- Benchmarks use b.N; compare with -benchmem.
- Test files are `_test.go`; external packages test public API.
- Prefer behavior assertions over mocking every collaborator.

Hold these in working memory while you code. Each bullet should map to something you can demonstrate with a three-line program or a failing compile error.

### Worked Go examples

Create a scratch module and type this yourself (do not paste blindly):

```
mkdir -p /tmp/gw-testing, && cd /tmp/gw-testing,
go mod init example.com/scratch
```

Then create `main.go`:

```
package clamp
```

```
import "testing"

func Clamp(v, lo, hi int) int {
    if v < lo {
        return lo
    }
    if v > hi {
        return hi
    }
    return v
}

func TestClamp(t *testing.T) {
    tests := []struct{ name string; v, lo, hi, want int }{
        {"mid", 5, 0, 10, 5},
        {"low", -1, 0, 10, 0},
    }
    for _, tt := range tests {
        t.Run(tt.name, func(t *testing.T) {
            if got := Clamp(tt.v, tt.lo, tt.hi); got != tt.want {
                t.Fatalf("got %d want %d", got, tt.want)
            }
        })
    }
}
```

Run it:

```
go run .
```

Expected output:

```
PASS (go test)
```

If your output differs, diff character-by-character — especially whitespace and types. Then change one line deliberately and predict the new output before running.

## Mini-lab (`go run` steps)

1. Convert a single assertion test into a table.
2. Add a fuzz target for a pure function.
3. Run `go test -bench=. -benchmem` on a tiny Sum.

Write down one sentence explaining *why* the output matches the code before you move on. Time-box the lab to 15–25 minutes; struggle is part of encoding the skill.

## Curriculum notes (expanded from the compact draft)

## Table-driven tests

```
func TestClamp(t *testing.T) {
    tests := []struct{
```

```
        name string
        v, lo, hi, want int
    }{
        {"mid", 5, 0, 10, 5},
        {"low", -1, 0, 10, 0},
    }
    for _, tt := range tests {
        t.Run(tt.name, func(t *testing.T) {
            if got := Clamp(tt.v, tt.lo, tt.hi); got != tt.want {
                t.Fatalf("got %d want %d", got, tt.want)
            }
        })
    }
}
```

## Fuzzing

```
func FuzzReverse(f *testing.F) {
    f.Add("go")
    f.Fuzz(func(t *testing.T, s string) {
        // property: reverse(reverse(s)) == s for runes...
    })
}
```

## Benchmarks

```
func BenchmarkSum(b *testing.B) {
    xs := make([]int, 1000)
    for i := 0; i < b.N; i++ {
        Sum(xs)
    }
}
```

Run: `go test -bench=. -benchmem ./...`

## Failure modes to watch while coding

- Giant tests without cases (hard to see failures)
- Testing implementation details instead of behavior
- Ignoring flaky time/network tests — inject clocks and interfaces

When a test fails, ask which failure mode you hit before changing code at random.

## Idioms vs Python pitfalls

- pytest fixtures ≈ careful constructor helpers — keep tests plain.
- No assert statement — use t.Fatalf / testify if your team allows.
- Time/network flakiness: inject clocks and interfaces like in Go.

When you catch yourself reaching for a Python habit, pause and rewrite the Go way once on purpose — even if both would “work”. The goal is a Go-shaped intuition.

## Going deeper

A failing test name should tell you the behavior broken without reading the body.

## Practice checklist

Before the exercise, confirm you can:

1. Explain “Testing, Table Tests, Fuzz, and Benchmarks” aloud in under two minutes without notes.
2. Re-type the worked example from memory and match the expected output.
3. Name the Go standard library packages touched by this lesson.
4. List one production bug this lesson’s rules prevent.
5. Contrast the Go approach with the Python instinct listed above.
6. Write a three-line cheat sheet: one idiom, one pitfall, one command to remember.

If any item fails, re-run the mini-lab once more, then attempt the exercise tests.

## Transfer to production

Ask where this shows up outside toys: config loading, API handlers, data pipelines, CI scripts, or on-call debugging. Write one concrete two-sentence scenario tying the worked example to that scenario. Tomorrow, when you rewrite the exercise from memory, check whether the code still fits that framing.

## Common mistakes (especially from Python instincts)

- Giant tests without cases (hard to see failures)
- Testing implementation details instead of behavior
- Ignoring flaky time/network tests — inject clocks and interfaces

## Exercises

Work in `exercises/12-testing/`. Read that folder’s `README.md` before you code.

### What you will build

Make table-driven tests pass by implementing the stub under test.

### Study loop

- **A (Drill):** make unit tests pass with the simplest correct code.
- **B (Apply):** extend with an edge case called out in the exercise README.
- **C (Challenge):** rewrite your solution from memory the next day without looking.

**Do not open `solutions/12-testing/` until you have an honest failing attempt.**

## Self-check

Answer without scrolling up:

1. Why table tests scale better?
2. What does b.N mean?

3. When is fuzzing valuable?

## Further reading

- https://go.dev/doc/tutorial/add-a-test
- https://go.dev/security/fuzz/